

MGS 616_HW7_Group13

Group Members:

Anuja Patil

Saumya Kumari

Nimisha Nooti

Tanweer Ashif

Satyam

- 1. A neural net typically starts out with random values for bias and weights; hence, it produces essentially random predictions when presented with its first case. What is the key ingredient by which the net evolves to produce a more accurate prediction?**

- Initial Randomization:**

- The weights and biases for the network are initialized at random values, typically small positive or negative numbers next to zero. Having different random initialization this guarantees that neurons in the network are asymmetric and can learn different features. At this point, the network's predictions are random and have no meaningful relationship with the target values.

- Forward Propagation:**

- The input data travels across the network from an input layer to an output layer. To do this: at every neuron, a weighted sum of the inputs is taken and then a bias is added. The next step is to pass this statement through an activation function (e.g., sigmoid, ReLU), which adds non-linearity and allows the network to learn complicated relationships in the data. And the last step from the network is to give output as predicted value for the given input.

- Loss Calculation:**

- The difference between the model predicted output and actual target is calculated by what is called a loss function (e.g, Mean Squared Error for regression, Cross-Entropy for classification etc.). The loss function gives a scalar value that is single number summing up the error in predictions made by the network. The next step serves to

measure how poorly the network is doing so far and establish a basis for improvements.

- **Backpropagation:**

- Backward propagation of error(loss), pass through the network to see how much each weight and each bias, contributed to the same error. We calculate the gradients (partial derivatives) of the loss with respect to all of the weights and biases using the chain rule. After calculating the loss, gradients derive information on how much and in which direction we should adjust our weights to lower the loss. In a neural network, backpropagation is the plot and heart of how networks get trained, allowing the model to “learn” from its mistakes.

- **Weight Update:**

- Updates weights and biases, these are updated using the gradients from backpropagation. The optimization algorithm (Stochastic Gradient Decent (SGD) or Adam) updates the parameters in the direction that reduces the loss. This hyperparameter is known as the learning rate which determines how big these changes are. Through this iterative process, the network gradually learns to make correct predictions.

- **Iteration:**

- The process, from forward propagation to weight updates, is repeated numerous times (epochs). An epoch is passing the complete dataset through the network and adjusting weight according to error. The network learns (over time) how to minimize that loss and thus do a better job of making predictions. When it is done right, the model converges to a point that will perform well on both train and test data. To avoid overfitting, regularization techniques and validation checks are frequently employed.

2. Create a small Python program to illustrate one pass through a simple neural network. (Randomly generate initial weight values.)

Without Normalization

Code Snippet:

Without Normalisation

```
: # Defined activation function (sigmoid)
def sigmoid(x):
    return 1 / (1 + np.exp(-x))

# Randomly generate initial weights and biases
np.random.seed(42)
weights = np.random.rand(2, 1) # 2 inputs, 1 output
bias = np.random.rand(1)

# Extract inputs from the DataFrame
inputs = df[["Years", "Salary"]].values

# Forward pass: Compute weighted sum and apply activation
weighted_sum = np.dot(inputs, weights) + bias
predictions = sigmoid(weighted_sum)

df["Predictions"] = predictions

print("Randomly initialized weights:")
print(weights)

print("\nRandomly initialized bias:")
print(bias)

print("\nData with Predictions:")
print(df)
```

Data with Predictions:

	Years	Salary	UsedCredit	Predictions
0	4	43	0	1.0
1	18	65	1	1.0
2	1	53	0	1.0
3	3	95	0	1.0
4	15	88	1	1.0
5	6	112	1	1.0

In the absence of normalization, the Salary feature would be comparatively larger than all other features and its corresponding weight has a high magnitude too, which causes it to dominate the weighted sum. As a result, the it produces very high weighted sums that cause the output of sigmoid function to saturate at 1.0 and consequently predicts 1.0 for all inputs as shown above.

With Normalization

Code Snippet

With Normalisation

```
: # Define activation function (sigmoid)
def sigmoid(x):
    return 1 / (1 + np.exp(-x))

# Randomly generate initial weights and biases
np.random.seed(42)
weights = np.random.rand(2, 1) # 2 inputs, 1 output
bias = np.random.rand(1)

# Extract inputs from the DataFrame
inputs = df[["Years", "Salary"]].values

# Normalize inputs to bring values to a smaller range
inputs = inputs / np.max(inputs, axis=0)

# Forward pass: Compute weighted sum and apply activation
weighted_sum = np.dot(inputs, weights) + bias
predictions = sigmoid(weighted_sum)
|
df["Predictions"] = predictions

# Display the results
print("Randomly initialized weights:")
print(weights)

print("\nRandomly initialized bias:")
print(bias)

print("\nData with Predictions (Normalized Inputs):")
print(df)
```

Data with Predictions (Normalized Inputs):

	Years	Salary	UsedCredit	Predictions
0	4	43	0	0.764989
1	18	65	1	0.840008
2	1	53	0	0.769004
3	3	95	0	0.832134
4	15	88	1	0.857062
5	6	112	1	0.859066

The input features (Years and Salary) were normalized to ensure all values are between 0–1. This guarantees, that both of these features contribute with the same proportion to the neural network calculations. As Salary is in a wider range, normalizing them would make our predictions biased and the training slower. It also normalizes your activations making your network more robust and faster in forward propagation.